

STM32 通用定时器

PWM 输出 • 输入捕获 • 输出比较

芯片平台	STM32F407 (Cortex-M4, 168MHz)
定时器范围	通用定时器 TIM2~TIM5、TIM9~TIM14
核心功能	PWM 输出 / 输入捕获 / 输出比较 / 编码器接口
HAL 库版本	STM32 HAL (CubeMX 生成框架)
内容涵盖	时基原理 / PWM / 输入捕获 / 输出比较 / DMA / 实战

2026-06-14

第一章 定时器概述与时基单元

1.1 STM32F407 定时器分类

类型	定时器	位数	主要功能
基本定时器	TIM6、TIM7	16位	仅计数/溢出，触发DAC，无通道
通用定时器	TIM2~TIM5 TIM9~TIM14	TIM2/5为32位 其余16位	PWM、输入捕获、输出比较、编码器
高级定时器	TIM1、TIM8	16位	通用功能 + 互补输出 + 死区 + 刹车（电机控制）

★ 本讲解聚焦通用定时器（TIM2~TIM5最常用，TIM2/TIM5为32位，可计数到42亿）。

1.2 时基单元三大寄存器

定时器的核心是时基单元（Time Base Unit），由三个关键寄存器构成：

寄存器	全称	作用
PSC	Prescaler 预分频器	对输入时钟分频，决定计数频率。实际分频 = PSC + 1
CNT	Counter 计数器	当前计数值，每个计数时钟自增/自减
ARR	Auto-Reload Register 自动重装值	计数上限，CNT 达到 ARR 时溢出并重装

1.3 定时器频率与溢出时间计算

计数频率 $CK_CNT = TIM_CLK / (PSC + 1)$

溢出周期 $T = (ARR + 1) / CK_CNT$
 $= (ARR + 1) \times (PSC + 1) / TIM_CLK$

其中 TIM_CLK（定时器时钟）规则：

· 若 APB1/APB2 预分频系数 = 1，则 $TIM_CLK = APBx$

· 若 APB1/APB2 预分频系数 > 1，则 $TIM_CLK = APBx \times 2$

STM32F407 默认：

APB1 = 42MHz（预分频4），故 TIM2~7 时钟 = $42 \times 2 = 84MHz$

APB2 = 84MHz（预分频2），故 TIM1/8/9/10/11 时钟 = $84 \times 2 = 168MHz$

□ 初学者最易错点：TIM2~TIM7 挂在 APB1 上，但由于 APB1 预分频>1，定时器时钟是 APB1 的 2 倍 = 84MHz，不是 42MHz！

1.4 定时 500ms 举例（TIM3，84MHz）

目标定时 500ms：设 PSC=8400-1（分频后 = 10kHz），ARR=5000-1：

$$T = (ARR+1) \times (PSC+1) / TIM_CLK$$
$$= 5000 \times 8400 / 84000000$$
$$= 42000000 / 84000000$$
$$= 0.5 \text{ s} = 500\text{ms} \quad \square$$

1.5 三种计数模式

计数模式	行为	溢出事件
向上计数 (Up)	0 → ARR, 溢出后回到 0	CNT=ARR 时产生更新事件 UEV
向下计数 (Down)	ARR → 0, 下溢后回到 ARR	CNT=0 时产生更新事件 UEV
中央对齐 (Center)	0→ARR→0 往复计数	上溢和下溢各产生一次 UEV, PWM对称

第二章 PWM 输出

2.1 PWM 基本原理

PWM (Pulse Width Modulation, 脉宽调制) 通过改变方波的占空比来等效调节输出能量，广泛用于电机调速、LED 调光、舵机控制、DAC 模拟输出等。

STM32 PWM 由「ARR 决定周期、CCR 决定占空比」实现：定时器计数 0→ARR 为一个 PWM 周期，通过比较 CNT 与 CCR (捕获/比较寄存器) 的大小输出高/低电平。

2.2 PWM 模式 1 与模式 2

PWM 模式	向上计数时的输出	占空比关系
PWM 模式 1 (PWM1)	CNT < CCR 时输出有效(高), CNT ≥ CCR 时输出无效(低)	占空比 = CCR / (ARR+1)
PWM 模式 2 (PWM2)	CNT < CCR 时输出无效(低), CNT ≥ CCR 时输出有效(高)	占空比 = 1 - CCR/(ARR+1)

★ 最常用 PWM 模式1 + 有效电平为高。此时占空比 = CCR/(ARR+1)，CCR 越大占空比越高，直观易理解。

2.3 PWM 频率与占空比公式

PWM 频率 $f = \text{TIM_CLK} / [(\text{PSC}+1) \times (\text{ARR}+1)]$

占空比 $D = \text{CCR} / (\text{ARR}+1)$ (PWM模式1, 高电平有效)

改变占空比 → 运行时修改 CCR (无需停止定时器)

改变频率 → 修改 ARR (同时要等比例调整 CCR 保持占空比)

2.4 PWM 配置实战 (TIM3_CH1, PA6, 1kHz, 50%)

TIM3 时钟 84MHz, 目标 1kHz: PSC=84-1 (计数频率1MHz) , ARR=1000-1 (周期1ms=1kHz) 。占空比 50% 即 CCR=500。

```
TIM_HandleTypeDef htim3 = {0};
TIM_OC_InitTypeDef sConfigOC = {0};

// --- 时基配置 ---
htim3.Instance = TIM3;
htim3.Init.Prescaler = 84 - 1; // 84MHz/84 = 1MHz
htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
htim3.Init.Period = 1000 - 1; // ARR: 周期1ms → 1kHz
htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
```

```
HAL_TIM_PWM_Init(&htim3);

// --- PWM 通道配置 ---
sConfigOC.OCMode = TIM_OCMODE_PWM1; // PWM 模式1
sConfigOC.Pulse = 500; // CCR: 占空比 500/1000 = 50%
sConfigOC.OCpolarity = TIM_OCPOLARITY_HIGH; // 高电平有效
sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC, TIM_CHANNEL_1);

// --- 启动 PWM 输出 ---
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
```

□ 别忘了在 HAL_TIM_MspPostInit 中把 PA6 配置为复用推挽 AF2 (TIM3) 功能, 否则引脚无 PWM 波形输出。CubeMX 会自动生成此配置。

2.5 运行时动态调节占空比

```
// 方法1: 宏直接写 CCR 寄存器 (最快)
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 250); // 改为 25%

// 方法2: 呼吸灯效果示例
void breathing_led(void) {
    for (uint16_t ccr = 0; ccr < 1000; ccr++) {
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, ccr);
        HAL_Delay(1); // 逐渐变亮
    }
    for (uint16_t ccr = 1000; ccr > 0; ccr--) {
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, ccr);
        HAL_Delay(1); // 逐渐变暗
    }
}
```

2.6 PWM 控制舵机示例 (SG90, 50Hz)

舵机要求 50Hz (20ms周期) PWM, 0.5ms~2.5ms 高电平对应 0°~180°:

```
// 50Hz: PSC=84-1 (1MHz) , ARR=20000-1 (20ms周期)
// 1us 一个计数, CCR 数值即高电平微秒数
htim3.Init.Prescaler = 84 - 1;
htim3.Init.Period = 20000 - 1;

// 角度转 CCR: 0.5ms(500)→0° 2.5ms(2500)→180°
uint16_t angle_to_ccr(float deg) {
    return (uint16_t)(500 + (deg / 180.0f) * 2000);
}
```

```
}  
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, angle_to_ccr(90)); // 转到90°
```

第三章 输入捕获

3.1 输入捕获原理

输入捕获（Input Capture）用于测量外部信号的频率、周期、脉宽、占空比。当捕获通道引脚检测到指定边沿（上升/下降）时，硬件自动将当前 CNT 值锁存到 CCR 寄存器，并可触发中断。通过记录两次边沿的 CNT 差值即可算出时间间隔。

3.2 输入捕获能测什么？

测量目标	方法
信号周期/频率	测量两个相邻上升沿之间的 CNT 差值 → 周期；取倒数 → 频率
高电平脉宽	测上升沿到下降沿的差值
占空比	同时测周期和高电平脉宽，占空比 = 高电平时间 / 周期

3.3 简单频率测量（单通道，上升沿捕获）

```
// TIM2_CH1 捕获, PSC=84-1 → 1MHz (1us 分辨率)
TIM_IC_InitTypeDef sConfigIC = {0};

htim2.Instance = TIM2;
htim2.Init.Prescaler = 84 - 1; // 1MHz 计数
htim2.Init.Period = 0xFFFFFFF; // TIM2 是32位, 给最大值
htim2.Init.CounterMode= TIM_COUNTERMODE_UP;
HAL_TIM_IC_Init(&htim2);

sConfigIC.ICPolarity = TIM_ICPOLARITY_RISING; // 上升沿捕获
sConfigIC.ICSelection = TIM_ICSELECTION_DIRECTTI;
sConfigIC.ICPrescaler = TIM_ICPSC_DIV1; // 不分频
sConfigIC.ICFilter = 0;
HAL_TIM_IC_ConfigChannel(&htim2, &sConfigIC, TIM_CHANNEL_1);

// 启动捕获 + 中断
HAL_TIM_IC_Start_IT(&htim2, TIM_CHANNEL_1);

// 捕获中断回调：计算两次上升沿间隔
volatile uint32_t g_freq = 0;
static uint32_t cap1 = 0, cap2 = 0;
static uint8_t cap_state = 0;

void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim) {
    if (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1) {
```



```
if (cap_state == 0) {
    cap1 = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
    cap_state = 1;
} else {
    cap2 = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
    uint32_t period = cap2 - cap1; // 32位计数器无需处理溢出
    if (period != 0)
        g_freq = 1000000UL / period; // 1MHz计数 → Hz
    cap_state = 0;
}
}
```

★ TIM2/TIM5 是 32 位计数器，测量低频信号时不易溢出，是输入捕获首选。16 位定时器（TIM3/4）测低频时需在中断中累计溢出次数。

3.4 PWMI 模式：测量占空比（双通道）

PWMI（PWM Input）模式用两个通道捕获同一个输入信号：CH1 捕获上升沿测周期，CH2 捕获下降沿测高电平时间，硬件自动完成，一次中断同时得到周期和占空比。

```
// TIM3_CH1 作为输入，PWMI 模式同时使用 CH1/CH2
TIM_IC_InitTypeDef sConfigIC = {0};

// CH1：上升沿捕获，直接连到 TI1，测周期
sConfigIC.ICPolarity = TIM_ICPOLARITY_RISING;
sConfigIC.ICSelection = TIM_ICSELECTION_DIRECTTI;
sConfigIC.ICPrescaler = TIM_ICPSC_DIV1;
sConfigIC.ICFilter = 0;
HAL_TIM_IC_ConfigChannel(&htim3, &sConfigIC, TIM_CHANNEL_1);

// CH2：下降沿捕获，间接连到 TI1，测高电平宽度
sConfigIC.ICPolarity = TIM_ICPOLARITY_FALLING;
sConfigIC.ICSelection = TIM_ICSELECTION_INDIRECTTI;
HAL_TIM_IC_ConfigChannel(&htim3, &sConfigIC, TIM_CHANNEL_2);

// 从模式：复位模式，每个上升沿自动复位 CNT
TIM_SlaveConfigTypeDef sSlave = {0};
sSlave.SlaveMode = TIM_SLAVEMODE_RESET;
sSlave.InputTrigger = TIM_TS_TI1FP1;
HAL_TIM_SlaveConfigSynchro(&htim3, &sSlave);

HAL_TIM_IC_Start_IT(&htim3, TIM_CHANNEL_1); // 周期中断
```

```
HAL_TIM_IC_Start(&htim3, TIM_CHANNEL_2); // 高电平宽度

// 中断中读取：
// 周期 = CCR1 + 1
// 高电平 = CCR2 + 1
// 占空比 = CCR2 / CCR1 x 100%
```

★ PWMI 模式 + 从模式复位是测量占空比的标准硬件方案，CNT 由上升沿自动清零，读 CCR1 得周期、读 CCR2 得高电平，无需软件计算两次差值。

3.5 输入捕获滤波与边沿

ICFilter（输入滤波器）：对输入信号采样多次确认后才认为有效边沿，过滤毛刺。取值 0~15，值越大滤波越强但响应越慢。

ICPrescaler（输入分频）：可每 1/2/4/8 个边沿才捕获一次，用于高频信号降频。

第四章 输出比较

4.1 输出比较原理

输出比较 (Output Compare) 将 CNT 与 CCR 持续比较，当两者相等时，按设定方式改变输出引脚电平并可触发中断/DMA。PWM 实际上是输出比较的一种特殊模式，但输出比较还可实现更灵活的波形控制。

4.2 输出比较的几种模式

输出比较模式	CNT=CCR 时的动作
冻结 (Frozen)	CCR 与 CNT 比较不影响输出，仅用作时基/触发
有效电平 (Active)	匹配时输出强制为有效电平 (高)
无效电平 (Inactive)	匹配时输出强制为无效电平 (低)
翻转 (Toggle)	匹配时输出电平翻转，常用于产生固定频率方波
PWM 模式1/2	用于生成 PWM (见第二章)
强制有效/无效	软件强制输出电平，不依赖 CNT

4.3 翻转模式产生方波 (TIM4_CH1)

翻转 (Toggle) 模式：每次 CNT=CCR 输出翻转一次，输出方波频率 = 更新频率 / 2。可在 CCR 不变时产生固定频率方波。

```
TIM_OC_InitTypeDef sConfigOC = {0};

// 时基: 84MHz/84=1MHz, ARR=1000 → 更新频率 1kHz
htim4.Instance = TIM4;
htim4.Init.Prescaler = 84 - 1;
htim4.Init.Period = 1000 - 1;
htim4.Init.CounterMode= TIM_COUNTERMODE_UP;
HAL_TIM_OC_Init(&htim4);

sConfigOC.OCMode = TIM_OCMODE_TOGGLE; // 翻转模式
sConfigOC.Pulse = 500; // CCR: 匹配点
sConfigOC.OCpolarity = TIM_OCPOLARITY_HIGH;
HAL_TIM_OC_ConfigChannel(&htim4, &sConfigOC, TIM_CHANNEL_1);

HAL_TIM_OC_Start(&htim4, TIM_CHANNEL_1);

// 输出 500Hz 方波 (每个1ms周期翻转一次 → 2ms完整周期)
```

4.4 输出比较中断——精确定时动作

输出比较中断可用于在一个 PWM/计数周期内的精确时刻执行动作（如多路时序控制、软件 DAC、精确延时触发）：

```
// 比较匹配中断回调
void HAL_TIM_OC_DelayElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1) {
        // CNT 到达 CCR1，执行定时动作
        uint32_t ccr = __HAL_TIM_GET_COMPARE(htim, TIM_CHANNEL_1);
        // 动态设置下一次比较点（在当前基础上加偏移）
        __HAL_TIM_SET_COMPARE(htim, TIM_CHANNEL_1, ccr + 200);
    }
}

// 启动带中断的输出比较
HAL_TIM_OC_Start_IT(&htim4, TIM_CHANNEL_1);
```

4.5 PWM、输入捕获、输出比较对比

功能	信号方向	CNT与CCR关系	典型用途
PWM 输出	输出	CNT 与 CCR 比较决定占空比	电机调速、LED调光、舵机
输出比较	输出	CNT=CCR 时改变电平/触发中断	方波生成、时序控制、软件DAC
输入捕获	输入	边沿到来时锁存 CNT 到 CCR	测频率、周期、脉宽、占空比

★ 三者底层都基于「CNT 计数器 + CCR 捕获/比较寄存器」：
输出类（PWM/输出比较）是 CNT 比较 CCR 后改变引脚；
输入类（输入捕获）是引脚边沿把 CNT 存入 CCR。方向相反，硬件同源。

第五章 综合实战与进阶

5.1 PWM + DMA 输出任意波形

用 DMA 在每次更新事件自动把数组中的值写入 CCR，可输出任意波形（如正弦表、SPWM）：

```
// 正弦波 PWM 表（一个周期 50 点）
uint16_t sine_table[50]; // 预先用 sin 计算填充 0~ARR 范围

void sine_pwm_init(void) {
    for (int i = 0; i < 50; i++) {
        sine_table[i] = (uint16_t)((sinf(2*3.14159f*i/50) + 1.0f)
                                   * 0.5f * 1000); // 映射到 0~1000
    }
    // 每次更新事件 DMA 把下一个值写入 CCR1
    HAL_TIM_PWM_Start_DMA(&htim3, TIM_CHANNEL_1,
                          (uint32_t*)sine_table, 50);
}
```

★ PWM+DMA 是产生 SPWM（正弦脉宽调制）驱动逆变器、生成音频信号的核心技术，CPU 全程不干预，输出极其平滑。

5.2 多通道同步 PWM（三相）

同一个定时器的 4 个通道共享 ARR（同频率），各自独立 CCR（不同占空比/相位），适合三相电机驱动、RGB 调光等需要同步的场景：

```
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1); // 相位A
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_2); // 相位B
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_3); // 相位C

__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 500); // 各通道独立占空比
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, 300);
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, 700);
```

5.3 通用定时器常见问题排查

故障现象	可能原因	解决方法
PWM 无输出	未调用 MspPostInit 配置 GPIO 复用	确认引脚配置为 AF 复用模式，复用号正确（查数据手册）
PWM 频率不对（差2倍）	误以为 TIM 时钟=APB1 实际是 APB1 x 2	TIM2~7 时钟 = 84MHz，重新计算 PSC/ARR

输入捕获测不准	16位计数器溢出未处理； 滤波/分频设置不当	用32位TIM2/5； 或中断中累计溢出次数
占空比无法到 100%	CCR=ARR 时仍有1个计数低电平	CCR 设为 ARR+1 可输出全高
改 ARR 后占空比变了	只改了 ARR 没按比例改 CCR	改频率时同步按比例调整 CCR
DMA PWM 不更新	未使能更新事件 DMA 请求	用 HAL_TIM_PWM_Start_DMA 且配置正确的 DMA 通道

5.4 关键 HAL API 速查

功能	HAL 函数
启动 PWM	HAL_TIM_PWM_Start(&tim, CH)
启动 PWM(中断)	HAL_TIM_PWM_Start_IT(&tim, CH)
启动 PWM(DMA)	HAL_TIM_PWM_Start_DMA(&tim, CH, buf, len)
设置占空比(CCR)	__HAL_TIM_SET_COMPARE(&tim, CH, value)
读取 CCR	__HAL_TIM_GET_COMPARE(&tim, CH)
启动输入捕获	HAL_TIM_IC_Start_IT(&tim, CH)
读捕获值	HAL_TIM_ReadCapturedValue(&tim, CH)
启动输出比较	HAL_TIM_OC_Start_IT(&tim, CH)
设置 ARR	__HAL_TIM_SET_AUTORELOAD(&tim, value)
读取 CNT	__HAL_TIM_GET_COUNTER(&tim;)

5.5 本讲总结

- 通用定时器三大功能本质统一于「CNT 计数 + CCR 比较/捕获」：
- PWM 输出：周期由 ARR 定，占空比由 CCR 定，用于功率/能量调节。
 - 输出比较：CNT=CCR 时改变电平或触发中断，用于精确时序与波形生成。
 - 输入捕获：外部边沿锁存 CNT 到 CCR，用于测量频率、周期、脉宽、占空比。
- 掌握 PSC/ARR/CCR 三个寄存器的关系，是理解 STM32 定时器一切高级功能的钥匙。